Methods of Combining Neural Networks and Genetic Algorithms

Talib S. Hussain
Queen's University
hussain@qucis.queensu.ca

1. Introduction

In the past decade, two areas of research which have become very popular are the fields of neural networks (NNs) and genetic algorithms (GAs). Both are computational abstractions of biological information processing systems, and both have captured the imaginations of researchers all over the world. In general, NNs are used as learning systems and GAs as optimisation systems, but as many researchers have discovered, they may be combined in a number of different ways resulting in highly successful adaptive systems. In this tutorial, a summary will be given of these combination methods. This summary is not meant to be exhaustive, but rather to be indicative of the type of research being conducted. For a more detailed discussion, see Yao (1993) and Schaffer et al. (1992).

The tutorial is broken into three sections. In the first section, a brief introduction to the foundations of neural networks and genetic algorithms is given. It is assumed that the participants have a basic understanding of both fields, and this introduction is designed as a short refresher. In the second section, a variety of approaches to integrating NNs and GAs are presented. In the final section, some of the key research issues are discussed.

1.1 Neural Networks

To set up the terminology for the rest of the paper, let us review the basics of a neural network. A neural network is a computational model consisting of a number of connected elements, known as neurons. A neuron is a processing unit that receives input from outside the network and/or from other neurons, applies a local transformation to that input, and provides a single output signal which is passed on to other neurons and/or outside the network. Each of the inputs is modified by a value associated with the connection. This value is referred to as the connection strength, or weight, and roughly speaking, represents how much importance the neuron attaches to that input source. The local transformation is referred to as the activation function and is usually sigmoidal in nature.

A typical neural network is capable of representing many functions, as proved by Komolgorov's Theorem, but finding the best network needed to solve a specific problem is a very open-ended problem. If the developer knows the exact solution method, then he can program the network structure explicitly. However, if the problem is very complex or has no known solution, the developer may not know what structure to give the network. To this end, most neural network models include a learning rule which can change the network's structure over the course of training to arrive at a good final solution. Back-propagation is the most popular learning rule.

1.2 Genetic Algorithms

A variety of computational models based on evolutionary processes have been proposed, and the most popular models are those known as genetic algorithms. A genetic algorithm has four main elements: the genetic code, a concise representation for an individual solution; the population, a number of individual solutions; the fitness function, an evaluation of the usefulness of an individual; and the propagation techniques, a set of methods for generating new individuals. The genetic algorithm works as follows. First, a population of individuals is generated by randomly selecting different genes. The fitness of each individual is then evaluated, and the propagation techniques are applied to highly fit individuals to generate a new population - the next generation. The cycle of evaluate and propagate continues until a satisfactory solution, hopefully optimal, is found.

In a typical genetic algorithm, the genetic code is a fixed-length bit string and the population is always a fixed size. The three most common propagation techniques are elitism, mutation and crossover. In elitism, the exact individual survives into the next generation. In mutation, a new individual is created from an old one by changing a small number of randomly selected bits in its gene. In crossover, a new individual is created from two old ones by randomly selecting a split point in their genes are creating a new gene with the left part from one parent and the right part from another. In any genetic algorithm, the two key aspects are the genetic representation and the fitness function. Together, these determine the type of problem which is being solved and the possible solutions which may be generated.

2. Combining NNs and GAs

2.1 Supportive and Collaborative

Researchers have combined NNs and GAs in a number of different ways. Schaffer et al. have noted

that these combinations can be classified into one of two general types - supportive combinations in which the NN and GA are applied sequentially, and collaborative combinations in which they are applied simultaneously.

In a supportive approach, the GA and the NN are applied to two different stages of the problem. The most common combination is to use a GA to pre-process the data set that is used to train a NN. For instance, the GA may be used to reduce the dimensionality of the data space by eliminating redundant or unnecessary features. Supportive combinations are not highly interesting since the GA and NN are used very independently and either can easily be replaced by an alternative technique. Some other possible combinations include: using a NN to select the starting population for the GA; using a GA to analyse the representations of a NN; and using a GA and NN to solve the same problem and integrating their responses using a voting scheme. (Schaffer et al.)

Alternatively, in a collaborative approach, the GA and NN are integrated into a single system in which a population of neural networks is evolved. In other words, the goal of the system is to find the optimal neural network solution. Such collaborative approaches are possible since neural network learning and genetic algorithms are both form of search. A neural network learning rule performs a highly constrained search to optimise the network's structure, while a genetic algorithm performs a very general population-based search to find an optimally fit gene. Both are examples of biased search techniques, and "any algorithm that employs a bias to guide its future samples can be mislead in a search space with the right structure. There is always an Achilles heal." (Schaffer et al, p. 4) The primary reason researchers have looked at integrating NNs and GAs is the belief that they may compensate for each other's search weaknesses.

2.2 Evolution of Connection Weights

A genetic algorithm can be applied to optimising a neural network in a variety of ways. Yao has indicated three main approaches.- the evolution of weights, the evolution of topology, and the evolution of learning rules. In each case, the GA's genetic code varies highly.

In the first, the GA is used as the learning rule of the NN. The genetic code is a direct encoding of the neural network, with each weight being represented explicitly. The population of the GA are all NNs with the same basic topology, but with different weight values. Mutation and crossover thus affect only the weights of the individuals. A key question in such system is whether to use binary weights or real-valued ones - the latter increases the search space greatly.

Using GAs instead of gradient descent algorithms to train the weights can result in faster and better convergence. Better still, since GAs are good at global search but inefficient at local finely tuned search, a hybrid approach combining GAs and gradient descent are attractive. (Yao)

2.3 Evolution of architectures

In the second approach, the GA is used to select general structural parameters and the neural learning is used separately to trained the network and determine its fitness. This includes evolution of both the topology (i.e., connectivity pattern) and activation functions of each node, although most work has concentrated on the former and little has been done on the latter.

In architecture evolution, the genetic code can be either a direct or indirect encoding of the network's topology. In a direct encoding, each connection is explicitly represented (e.g., a matrix where 1 indicate the presence of a connection and 0 indicates no connection). In an indirect encoding, important parameters of the network are represented and the details of the exact connectivity are left to developmental rules (e.g., specify the number of hidden nodes and assume full connectivity between layers).

In both cases, the exact neural network is not specified since the weights are determined by the initialisation routine and the network's learning algorithm. Thus, the evaluation of a gene is noisy since it is dependent upon the evaluation of the trained network, and the GA finds the best set of architectural parameters rather than the best neural network.

2.4 Evolution of learning rules

In the final approach, the GA is used similarly to the evolution of architecture, but a parametric representation of the network's learning rule is also encoded in the gene. The genetic coding of topology in this case is generally indirect.

Evolving learning rules does not refer simply to adapting learning algorithm parameters (e.g., learning rate, momentum, etc.) but to adapting the learning functions themselves. This is an area of research which has received little attention. "The biggest problem here is how to encode the dynamic behaviour of a learning rule into static genotypes. Trying to develop a universal representation scheme which can specify any kind of dynamic behaviours is clearly impractical let alone the prohibitive long computation time required to search such a learning rule space." (Yao, p. 214)

3. Issues

Collaborative combinations of NNs and GAs have sparked the interest of a great number of researchers because of their obvious analogy to natural

systems. A wide variety of systems have been developed and a number of research issues have been considered.

3.1 The Baldwin Effect

In general, one may wonder whether it really is of any use to have both neural learning and genetic search operating in the same system. Perhaps using just genetic search would work given enough time, or perhaps a very general neural learning technique would be sufficiently powerful. This is quite possibly true, but an observation from natural systems known as the Baldwin Effect provide a clearer answer.

The Baldwin Effect states that in an evolutionary system, successful genes can propagate faster, and in some cases only, if the individuals are capable of learning. This principle has been clearly demonstrated in an artificial evolutionary system by French & Messinger (1994). Thus, an evolutionary system with simple individuals which can learn is generally more successful than one with non-learning individuals and probably also better than a single highly complex learning individual.

3.2 Generalisation

In evolving a neural network, attention must be paid to the trade-off between evolutionary fitness and generalisation ability. In many tasks, the final network is trained on a small set of data and applied to a much larger set of data. The goal of the learning is actually to develop a neural network with the best performance on the entire problem and not just the training data. However, this can easily be overlooked during the development process.

Thus, one must be careful when evolving neural networks not to select for highly specialised, poorly generalising networks. This is especially true in problem areas which are highly dynamic.

3.3 Encoding Methods

The two main properties of an encoding of a neural network in a GA are its compactness and representation capability. A compact encoding is useful since the GA can then be efficiently applied to problems requiring large NN solutions. An encoding should be powerful enough to represent a large class of NNs or else the GA may not generate very good solutions. For instance, direct encoding is generally quite powerful in representation, but not compact, while parameterised encoding is compact, yet often represents a highly restrictive set of structures.

The discussion so far has focused on direct encoding and parametric encoding of neural network structure. Other possibilities also exist. In particular, grammatical encoding has recently received some attention. (Gruau, 1994) Grammar encoding is quite powerful since it is compact but can represent a great range of networks.

4. Conclusions

Neural networks and genetic algorithms are two highly popular areas of research, and integrating both techniques can often lead to highly successful learning systems. The participants of this tutorial are encouraged to try applying evolutionary neural network solutions, or even developing new combinations of their own.

References

French, R. & Messinger, A. (1994). "Genes, phenes and the Baldwin Effect: Learning and evolution in a simulated population," Artificial Life IV, 277-282.

Gruau, F. (1994) "Automatic definition of modular neural networks," Adaptive Behaviour, 3, 151-184.

Schaffer, D., Whitley, D. & Eshelman, L. (1992) "Combinations of Genetic Algorithms and Neural Networks: A survey of the state of the art," Proceedings of the International Workshop on Combinations of Genetic Algorithms and Neural Networks. D. Whitley and D. Schaffer (Eds.,) Los Alamitos, CA: IEEE Computer Society Press, 1-37.

Yao, X. (1993) "Evolutionary artificial neural networks" International Journal of Neural Systems, 4, 203-222.